

AI STUDY BUDDY API

Project Description:

AI Study Buddy is a web-based AI-powered learning assistant designed to help students understand, revise, and organize academic content efficiently. The system connects students with an AI service so that they can ask questions and receive relevant study assistance.

The application provides learning features including User Registration, User Login, Material Upload, AI Chat Assistance, Summarization, Flashcard Generation, Quiz Generation, and Study Plan support. Students can upload study materials and use AI processing to transform the content into concise summaries, revision flashcards, and practice quizzes.

The frontend is developed using HTML, CSS, and JavaScript. The backend uses Node.js and Express.js, while MongoDB is used for data storage. Generative AI is integrated through the configured Gemini API. The project is developed and tested using Visual Studio Code and Postman.

Scenarios

Scenario 1: A student creates an account, logs in, and accesses the Study Buddy dashboard to use the available learning features.

Scenario 2: A student uploads a study material and selects the summarization feature. The backend processes the request and returns an AI-generated summary.

Scenario 3: A student uploads learning material and generates flashcards. The system produces question-and-answer pairs that can be used for revision.

Scenario 4: A student generates a quiz from the uploaded material. The AI creates questions based on the study content for self-assessment.

Technical Architecture:

Description: AI Study Buddy uses a modular three-tier architecture consisting of the frontend, backend, and AI/database layers. The frontend provides the student interface. The Node.js and Express.js backend manages API requests, authentication, material processing, and communication with external services. MongoDB stores application data, while the Generative AI service processes academic questions and study materials to produce learning-oriented responses.

The main processing flow is Student → Frontend → Express.js Backend → MongoDB / Gemini AI Service → Backend → Frontend → Student.

Main Components

- User Interface
- Authentication Module
- Material Management Module
- AI Processing Module
- MongoDB Database
- REST API Layer

Pre-requisites:

- Node.js and npm installed
- Visual Studio Code or another JavaScript development environment
- MongoDB database/service and MongoDB Compass if required
- Postman for API testing
- Google Chrome, Microsoft Edge, or another modern web browser
- Basic knowledge of HTML, CSS, JavaScript, Node.js, Express.js, MongoDB, REST APIs, and JSON
- Configured Gemini Generative AI API access
- Git for version control

Project Workflow:

Milestone 1: Project Setup and Architecture

Activity 1.1: Define the project requirements and identify the major student-learning problems to be addressed.

Activity 1.2: Design the application architecture and define communication between frontend, backend, database, and AI service.

Activity 1.3: Set up the Node.js, Express.js, MongoDB, and development environment.

Activity 1.4: Configure the AI service and environment variables securely.

Milestone 2: User Authentication

Activity 2.1: Develop user registration functionality.

Activity 2.2: Develop login and authentication functionality.

Activity 2.3: Protect user-specific resources and study materials.

Milestone 3: Material Management

Activity 3.1: Develop material upload functionality.

Activity 3.2: Store and manage uploaded material information.

Activity 3.3: Prepare material content for AI-based processing.

Milestone 4: AI Learning Features

Activity 4.1: Implement study-material summarization.

Activity 4.2: Implement flashcard generation.

Activity 4.3: Implement quiz generation.

Activity 4.4: Implement AI chat assistance.

Milestone 5: Backend and API Development

Activity 5.1: Develop Express.js routes and controllers.

Activity 5.2: Connect APIs with MongoDB and the AI service.

Activity 5.3: Implement validation and error handling.

Milestone 6: Testing and Deployment

Activity 6.1: Test APIs using Postman.

Activity 6.2: Verify frontend-backend integration.

Activity 6.3: Run and demonstrate the application in the local development environment.

Milestone 1: Project Setup and Architecture

This milestone focuses on defining the AI Study Buddy requirements, selecting the technology stack, designing the system architecture, and preparing the development environment. The architecture separates presentation, application logic, data storage, and AI processing so that each major responsibility can be developed and tested independently.

Activity 1.1: Define Project Requirements

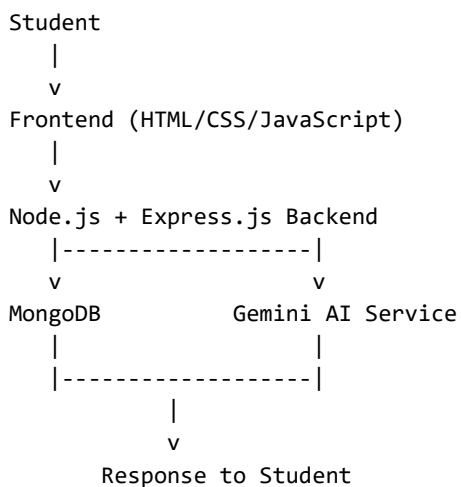
The requirements were identified around the needs of students who want quick academic assistance and easier revision. The project therefore includes authentication, material upload, AI-powered study-content generation, and interactive learning support.

The major functional requirements are:

- User Registration
- User Login
- Material Upload
- Summarize
- Flashcards
- Quiz
- AI Chat Assistance
- Study Plan

Activity 1.2: Define the Application Architecture

The application architecture defines how requests move from the student interface to the Express.js backend. The backend communicates with MongoDB for persistent data and with the configured Gemini AI service for AI-generated learning content. Responses are returned to the frontend and displayed to the student.



Milestone 1: Architecture and Development Environment

Activity 1.3: Set Up the Development Environment

The development environment is prepared using Node.js, npm, Visual Studio Code, MongoDB, and Postman. Node.js provides the runtime environment and npm manages the project dependencies. Express.js is used to implement the backend API layer. MongoDB provides persistent data storage.

The project can be started from the backend project directory after installing the required dependencies and configuring the environment variables.

Activity 1.4: Configure the AI Service

The AI service is configured through the backend so that API credentials are not exposed directly in the frontend. Environment variables are used to store sensitive configuration values.

A typical environment configuration is represented conceptually as:

```
GEMINI_API_KEY=your_api_key_here
```

The actual API key must be kept private and should not be committed to a public repository.

Development Tools

- Visual Studio Code – source-code development
- Node.js – JavaScript runtime
- npm – dependency management
- MongoDB – database
- Postman – API testing
- Git – version control

Milestone 1: Technical Architecture Details

The system is divided into functional layers to make development and maintenance easier.

Frontend Layer

The frontend provides the user interface for registration, login, material upload, AI interaction, and learning features. It sends requests to the backend API and displays the returned results.

Backend Layer

The Node.js and Express.js backend handles routing, request validation, authentication, database communication, material processing, and communication with the AI service.

Database Layer

MongoDB is used to persist user and application data. The exact collection names and field structure should follow the implementation used in the submitted project.

AI Processing Layer

The AI processing layer sends appropriate study-related prompts or material content to the configured Generative AI service and receives generated learning content such as summaries, flashcards, quizzes, and answers.

Security Considerations

- Keep API credentials in environment variables.
- Validate incoming requests.
- Protect authenticated endpoints.
- Do not expose private tokens in frontend source code.
- Handle external API failures gracefully.

Milestone 2: Registration and Login Implementation

Activity 2.1: Registration Module

The registration feature provides the entry point for a new student. The frontend collects the required details and sends them to the backend. The backend validates the information and stores the user record in MongoDB.

Activity 2.2: Login Module

The login feature allows an existing user to enter their credentials. The backend verifies the submitted information and establishes authenticated access to the application.

Activity 2.3: Validation and Error Handling

- Check that required registration fields are provided.
- Validate login input before database processing.
- Return appropriate error responses for invalid credentials.
- Prevent duplicate or invalid account records.
- Protect APIs that require authentication.

Expected Outcome

A registered student should be able to log in successfully and access the Study Buddy features, while invalid or unauthorized requests should be rejected with an appropriate response.

Milestone 2: User Authentication

User authentication provides controlled access to the Study Buddy application. Registration creates a user account, while login verifies the credentials and provides access to protected learning functionality.

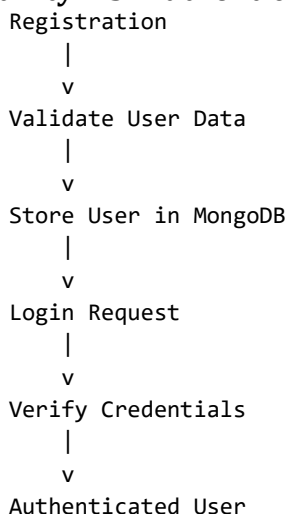
Activity 2.1: User Registration

The registration module collects the required user information, validates the input, and stores the account information in the database. Appropriate validation should be applied to prevent incomplete or invalid registration requests.

Activity 2.2: User Login

The login module accepts the registered user's credentials and verifies them against the stored account information. After successful authentication, the user can access the application features available to the authenticated session.

Activity 2.3: Authentication Flow





Access Study Buddy Features

Authentication testing should include valid credentials, invalid credentials, missing fields, duplicate registration attempts, and unauthorized access to protected resources.

Milestone 3: Material Upload

Material Upload is a core feature because the uploaded study content becomes the input for AI-powered learning functions.

Activity 3.1: Upload Study Material

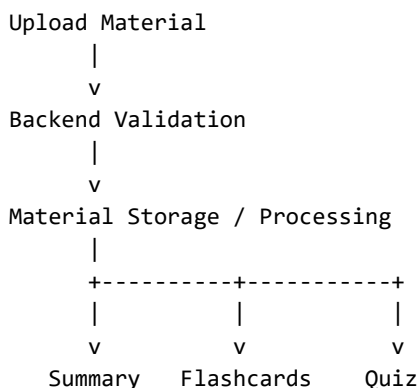
The student selects a study file and provides the required title or material information. The frontend sends the material to the backend using an API request.

Activity 3.2: Process and Store Material

The backend receives the upload request, validates the input, and manages the material information. The material is associated with the authenticated user so that it can be retrieved for subsequent learning operations.

Activity 3.3: Use Material for AI Features

After a material is available, the student can select operations such as Summarize, Flashcards, or Quiz. The backend retrieves the required material content and sends it to the AI processing layer.



Expected Outcome

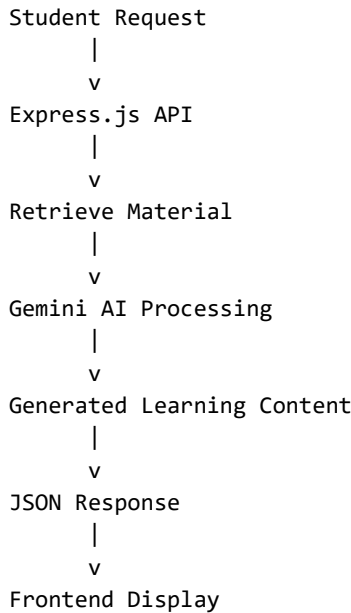
The uploaded material should be available for the student and should provide the input required for the supported AI learning features.

Milestone 4: AI Learning Features

This milestone focuses on transforming study material into useful learning resources through Generative AI. The demonstrated project APIs include summarization, flashcard generation, and quiz generation.

Activity 4.1: AI Processing Flow

When the student selects an AI learning feature, the frontend sends a request to the backend. The backend retrieves the relevant material, constructs the AI request, sends it to the configured Generative AI service, processes the returned response, and sends the result back to the frontend.



Activity 4.2: AI Response Handling

The backend should validate the AI response before returning it to the frontend. If the external AI service is unavailable or returns an invalid response, the backend should return an appropriate error response instead of exposing raw service errors to the student.

Milestone 4: Summarization

Activity 4.1: Generate Study Summary

The Summarize feature processes the selected study material and generates a concise explanation of the important content. This helps students review a larger amount of material in a shorter form.

API Endpoint Demonstrated

POST /api/materials/:id/summarize

The provided Postman demonstration shows the summarization request completing with HTTP Status 200 OK and returning a JSON response containing the generated summary.

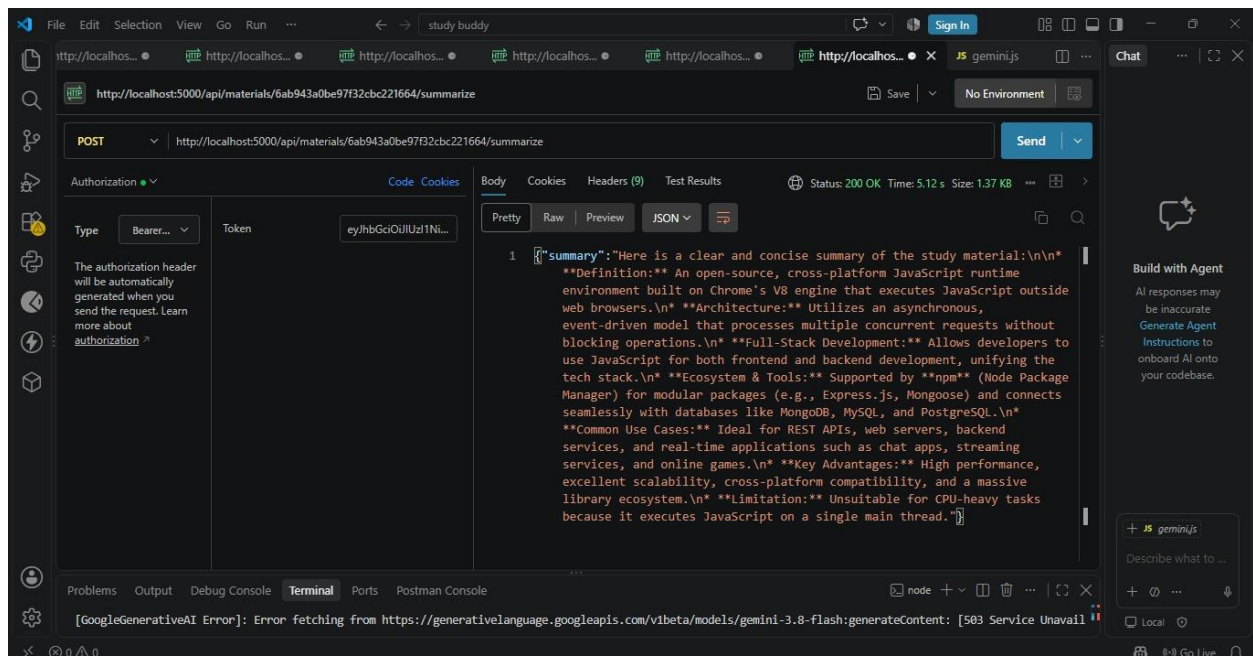


Figure: Postman demonstration of the Summarize API

Activity 4.2: Response Display

The returned summary is received by the frontend and displayed to the student in a readable format. The summary can be used as a quick revision resource.

Milestone 4: Flashcard Generation

Activity 4.2: Generate Flashcards

The Flashcards feature converts the selected study material into question-and-answer pairs. These generated flashcards can be used for active recall and revision.

API Endpoint Demonstrated

POST /api/materials/:id/flashcards

The provided Postman demonstration shows the Flashcards request completing with HTTP Status 200 OK and returning a JSON array containing generated questions and answers.

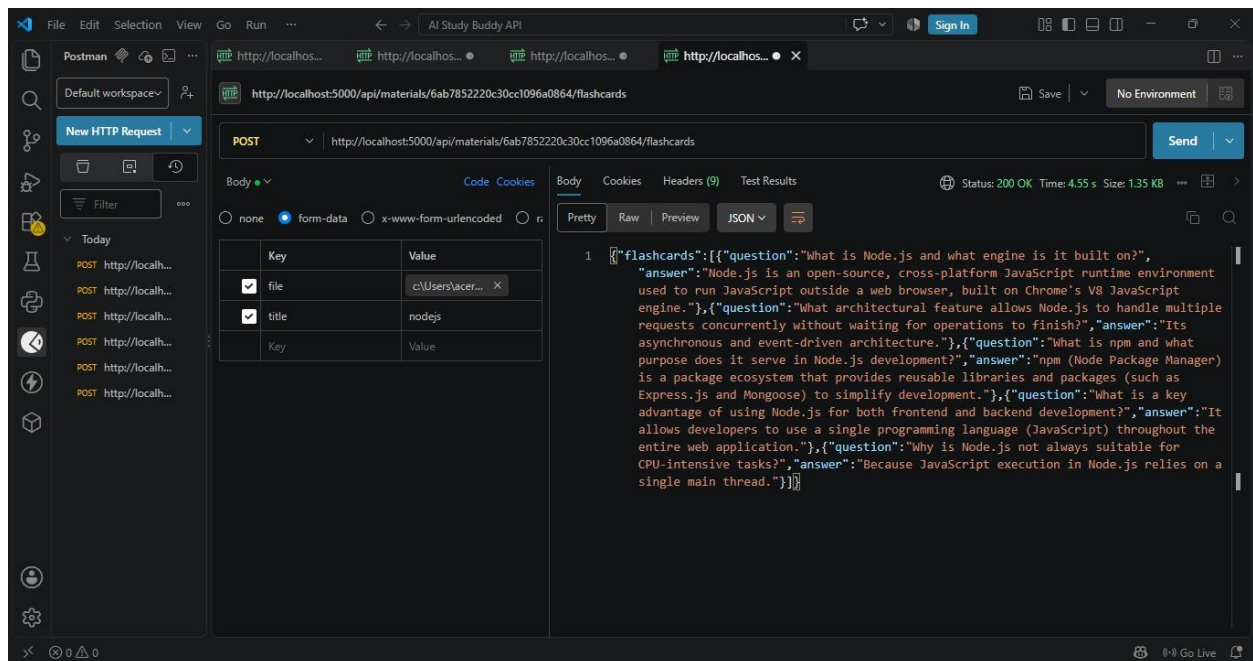


Figure: Postman demonstration of the Flashcards API

Expected Outcome

The system should return meaningful question-and-answer pairs derived from the uploaded study material and make them available to the student for revision.

Milestone 4: Quiz Generation

Activity 4.3: Generate Quiz

The Quiz feature uses the selected study material to generate questions for student practice and self-assessment. The AI output can contain questions and their corresponding answer information according to the implemented response structure.

API Endpoint Demonstrated

POST /api/materials/:id/quiz

The provided Postman demonstration shows the Quiz request completing with HTTP Status 200 OK and returning generated quiz questions in JSON format.

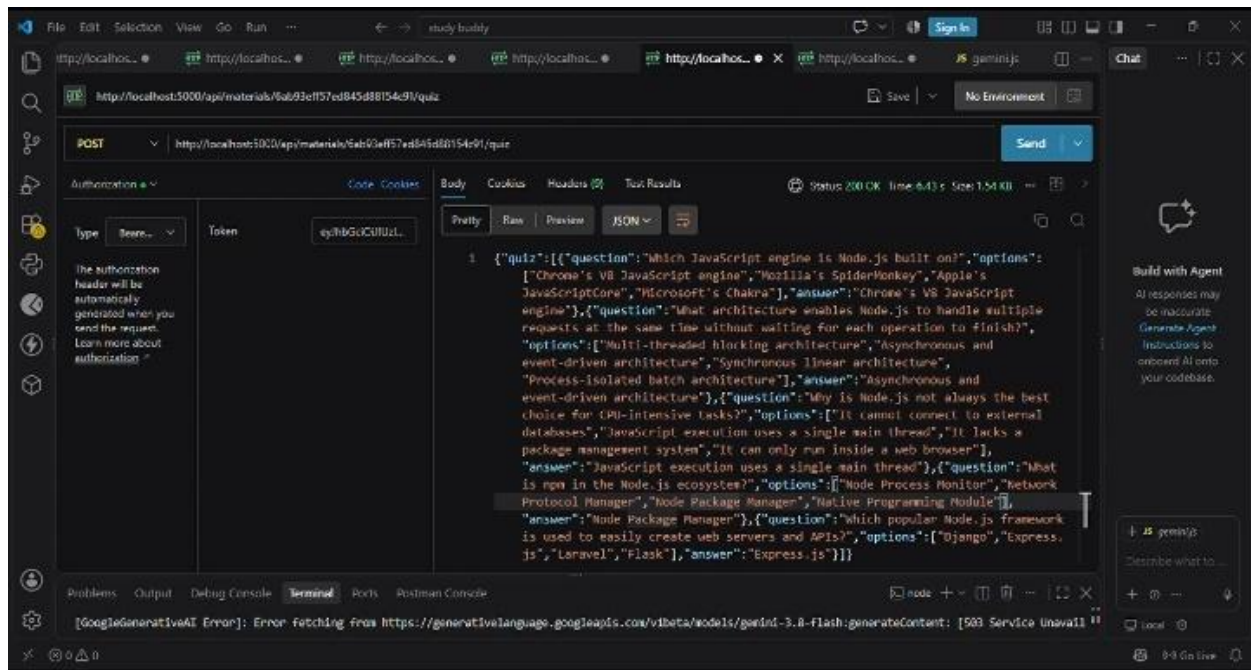


Figure: Postman demonstration of the Quiz API

Milestone 5: Backend and API Development

The backend acts as the central application layer connecting the frontend, database, and AI service.

Activity 5.1: Express.js API Layer

Express.js routes receive frontend requests and forward them to the appropriate application logic. The backend handles authentication, material operations, and AI-related requests.

Activity 5.2: Demonstrated API Endpoints

Method	Endpoint	Purpose
POST	/api/materials/:id/summarize	Generate a summary from a material
POST	/api/materials/:id/flashcards	Generate flashcards from a material
POST	/api/materials/:id/quiz	Generate quiz questions from a material

Activity 5.3: Error Handling

- Validate request parameters and required data.
- Handle database errors.
- Handle invalid material references.
- Handle AI-service failures.
- Return meaningful HTTP status codes and JSON error messages.

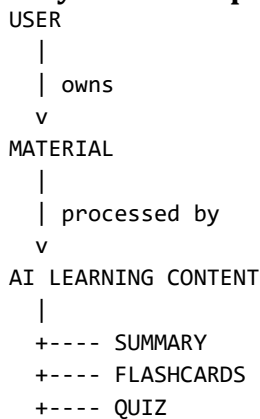
Milestone 6: Database Development

MongoDB is used as the database for the AI Study Buddy application. The database layer stores application information needed by the backend and allows the system to retrieve user and material data when required.

Activity 6.1: MongoDB Integration

The Express.js backend communicates with MongoDB to store and retrieve application records. The exact collections and field names should match the actual implementation in the submitted source code.

Activity 6.2: Conceptual ER/Data Relationship



Activity 6.3: Database Considerations

- Store only the data required by the application.
- Associate materials with the appropriate authenticated user.
- Validate data before storing it.
- Handle database connection failures gracefully.
- Protect database credentials through environment configuration.

Milestone 7: Frontend Development

The frontend provides a simple interface through which students interact with the AI Study Buddy system. It communicates with the backend using API requests and displays the responses returned by the server.

Activity 7.1: User Interface

- Registration interface
- Login interface
- Study dashboard
- Material upload interface
- Summary display
- Flashcards display
- Quiz interface
- AI chat interface
- Study plan interface

Activity 7.2: API Communication

The frontend sends HTTP requests to the Express.js API and processes the JSON responses. Successful responses are displayed in the appropriate feature area, while errors are handled through suitable messages.

Activity 7.3: User Experience

The interface should keep the main learning functions easy to find and should present AI-generated content in a clear format so that students can read, revise, and practise without unnecessary complexity.

Milestone 8: Testing and Local Deployment

Activity 8.1: API Testing with Postman

Postman is used to independently test the backend endpoints. This allows the team to verify request formats, authorization, response structures, status codes, and AI-generated output before relying on the frontend.

Observed API Demonstrations

- Summarize API – HTTP 200 OK demonstrated.
- Flashcards API – HTTP 200 OK demonstrated.
- Quiz API – HTTP 200 OK demonstrated.

Activity 8.2: Local Server

The supplied Postman screenshots show the application API running on a local Node.js server at localhost:5000.

Example local API base:
`http://localhost:5000`

Activity 8.3: Testing Scenarios

- Valid and invalid login requests
- Valid and invalid material uploads
- Summary generation
- Flashcard generation
- Quiz generation
- AI service unavailable/error response
- Database connectivity
- Unauthorized access attempts

Exploring the Website's Web Pages

Registration Page

Provides the form required for a new student to create an account.

Login Page

Allows registered students to authenticate and access the application.

Dashboard / Home Page

Provides access to the Study Buddy learning features and the student's available study materials.

Material Upload Page

Allows the student to select and upload a study material and provide the required title or information.

Summary Section

Displays the AI-generated concise summary of the selected study material.

Flashcards Section

Displays generated question-and-answer flashcards for revision.

Quiz Section

Displays generated quiz questions based on the selected material.

AI Chat Section

Provides a conversational interface for academic questions and AI-generated explanations.

Study Plan Section

Helps students organize learning activities through a structured study plan for preparation and revision.

Advantages and Future Enhancements

Advantages

- Provides quick academic assistance.
- Helps students simplify difficult study content.
- Automatically generates summaries.
- Creates flashcards for revision.
- Generates quizzes for self-assessment.
- Supports independent learning.
- Reduces the time required to prepare revision material.
- Provides an interactive learning experience.

Future Enhancements

- Voice-based AI interaction.
- Personalized study schedules.
- Student progress tracking.
- Performance analysis and learning analytics.
- Multilingual support.
- Subject-wise learning recommendations.
- Mobile application support.
- More advanced personalized learning features.

Scalability Considerations

- Support a larger number of simultaneous users.
- Optimize storage and retrieval of study materials.
- Improve AI response handling and performance.
- Strengthen authentication and application security.
- Introduce monitoring and performance analytics.

Conclusion

AI Study Buddy is an AI-powered web application designed to make academic learning more accessible and interactive. It combines a student-friendly frontend with a Node.js and Express.js backend, MongoDB data storage, and Generative AI processing.

The system supports important learning activities including user registration, login, material upload, summarization, flashcard generation, quiz generation, AI chat assistance, and study planning. The demonstrated Postman tests show successful HTTP 200 OK responses for the Summarize, Flashcards, and Quiz APIs.

By transforming uploaded study material into concise summaries, revision flashcards, and practice quizzes, AI Study Buddy provides a practical approach to self-learning and revision. Future versions can extend the platform with personalized schedules, progress tracking, voice interaction, multilingual support, and subject-specific recommendations.

